Introducción a IoT

Conceptos básicos de electrónica

TECNOLOGÍAS PARA LA AUTOMATIZACIÓN

Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

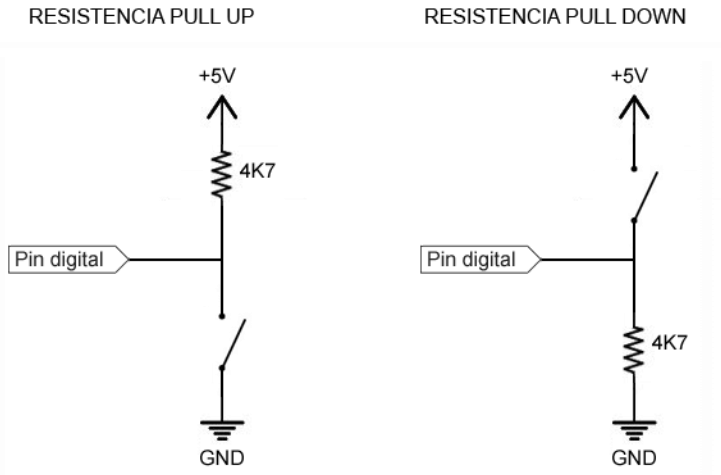
Cuando se intenta detectar actividad del mundo exterior usando un microcontrolador, las actividades más simples, son aquellas en las cuales uno necesita saber sólo una cosa: si algo es verdadero o falso.

¿Está en la habitación o no?.

¿Están tocando la mesa, o no?.

¿Está la puerta cerrada, o no?

¿Está pulsando el botón, o no?



ESP32 tiene conectada una Resistencia Interna de pullup o pulldown con cada PIN digital.

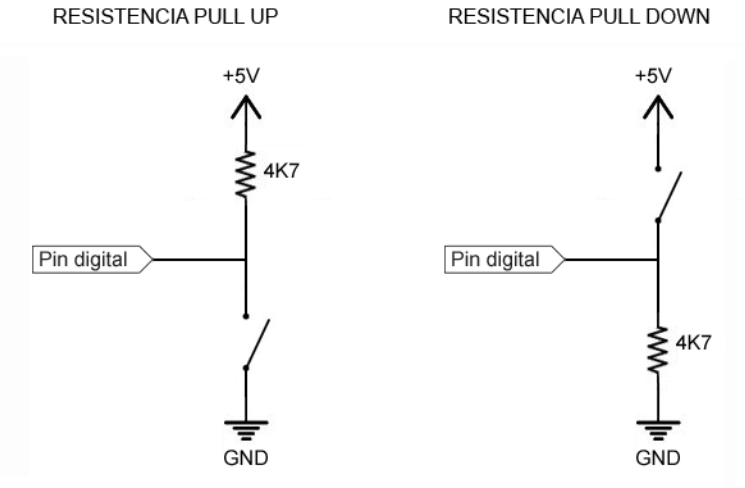
Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

Las entradas digitales o binarias de los microcontroladores tienen dos estados: On / Off (true o false; HIGH o LOW). Un valor de tension alto (5v o 3.3v) equivale a un “1” lógico, mientras que uno bajo equivale a un “0”.

Se pueden conectar:

- Pulsadores
- Sensores discretos
- Llaves / Interruptores



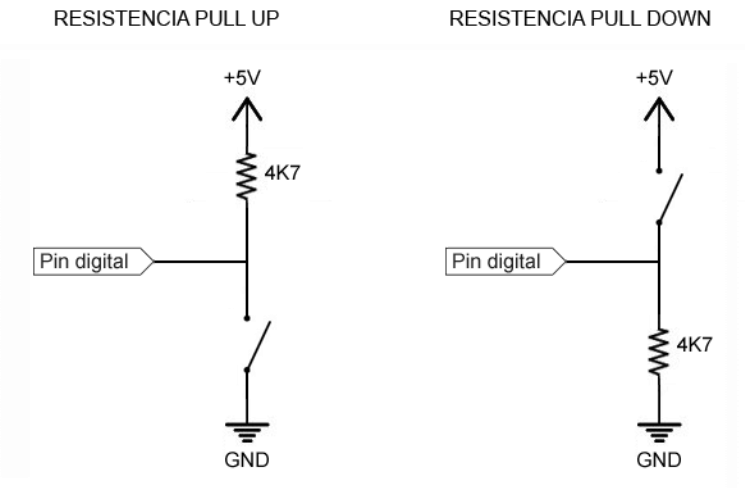
Arquitectura de Microcontroladores

Pull Up: el pin de entrada se conecta a 3.3V a través de una resistencia.
Sin actuar, cuando está abierto, el pin está a 3.3V. Cuando cierro el PIN está en cero.

Pull Down: A tierra a través de una resistencia. El pin está a Masa y mide 0V.

Cuando cierro el PIN está en 3.3V.

Útil cuando se están leyendo señales de entrada que pueden flotar entre estados lógicos, lo que puede provocar comportamientos inesperados en el circuito.



Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

- Las señales eléctricas medibles son cambios en el voltaje (no en corriente)
- Los dispositivos trabajan en 5v o 3.3v
- Es importante no sobrepasar la máxima tensión admitida por el dispositivo.
- Tampoco se recomienda usar una tensión menor, ya que puede darse que el sistema no reconozca el cambio de estado.

Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

- Los pines del controlador aceptan y/o producen muy poca corriente.
- Cuando se intenta leer un cambio eléctrico con corriente alta, esta se limita con una Resistencia.
- Cuando se desea controlar un circuito de alta corriente, se usa un transistor o un relé. Ambos permiten hacerlo con pequeños cambios de voltaje y mínima corriente.

Arquitectura de Microcontroladores

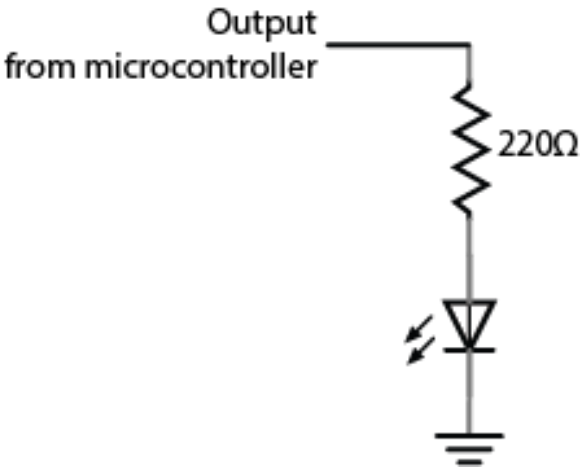
Puertos de Entrada y Salida Digitales

Entradas digitales permiten **detectar** actividades binarias.

Salidas digitales permiten **controlar** actividades que tienen dos estados. Con las salidas digitales se puede encender o apagar el dispositivo que se conecte al pin.

Se pueden conectar:

- Leds
- Pantallas LCD
- Transistores
- Relés



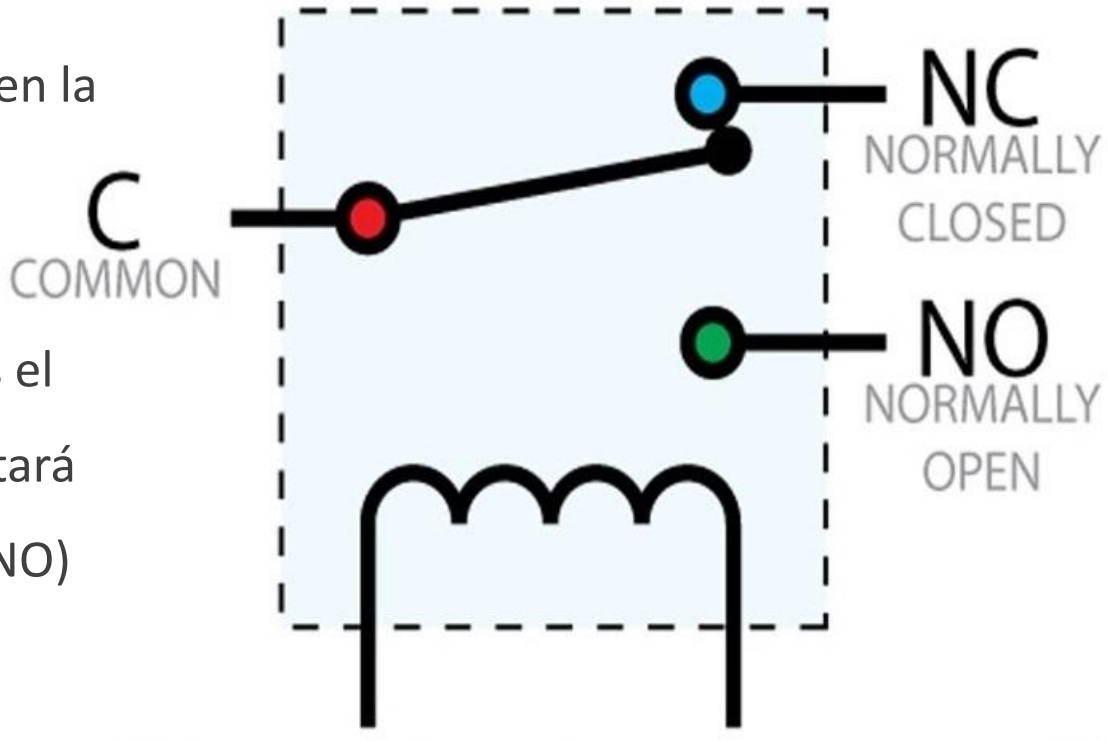
Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

Relé (Relay)

Sin actuar (Sin corriente en la bobina)

Según cómo conectemos el dispositivo, el circuito estará Cerrado (NC) o Abierto (NO)



Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

Relé (Relay)

Cuando actuamos sobre el relay.

(Le damos corriente a la

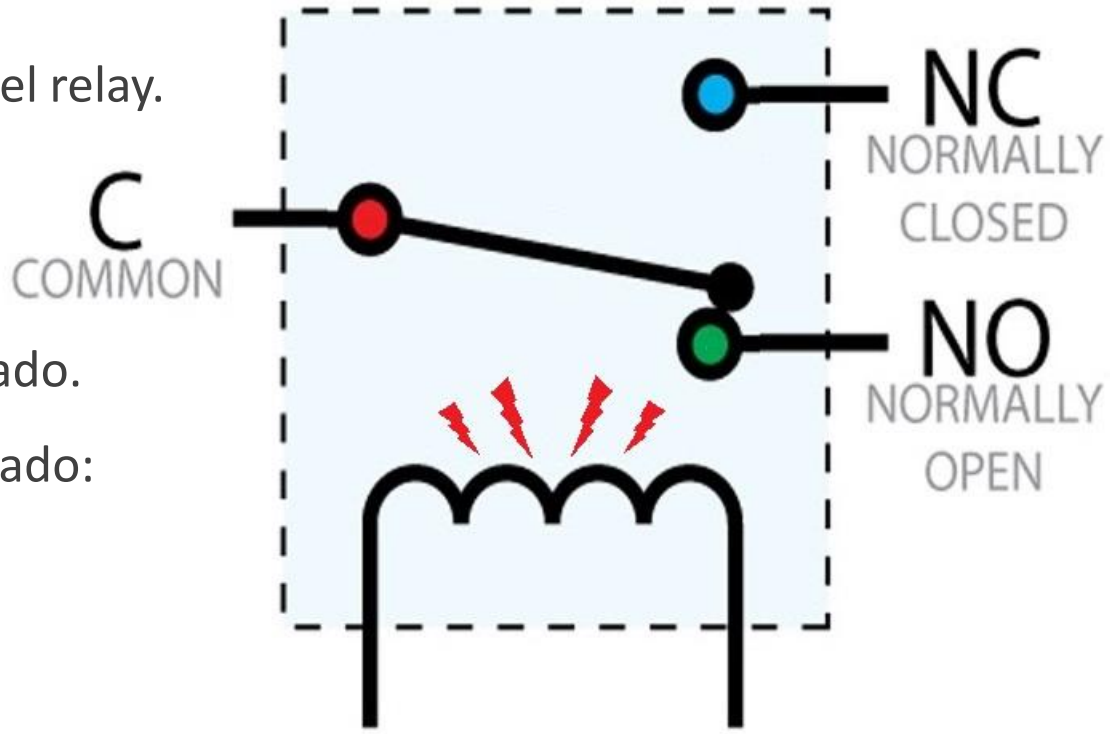
Bobina).

El circuito cambia de estado.

Si estaba conectado Cerrado:

se abre.

y viceversa.



Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

- Configuración

```
void setup()
{
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(11, INPUT);
    pinMode(9, INPUT_PULLUP);

  /* agregue código de setup aquí */
}
```

Arquitectura de Microcontroladores

Puertos de Entrada y Salida Digitales

- Uso

```
void loop()
{
    digitalWrite(LED_BUILTIN, HIGH);
    int a = digitalRead(11);
    bool b = digitalRead(9);

}
```

Arquitectura de Microcontroladores

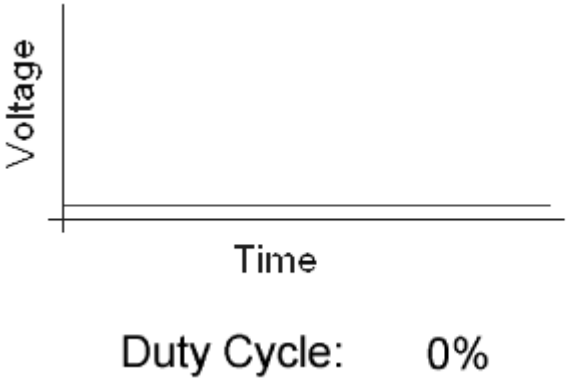
Puertos de Salida PWM (Pulse Width Modulation)

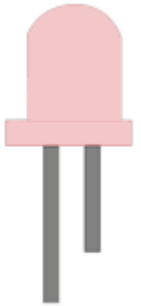
El valor medio de tension que se entrega a la carga es controlado por la rápida conmutación del voltaje que se entrega a la carga.

La frecuencia de conmutación es del orden de las decenas de kiloHertz

Se pueden conectar:

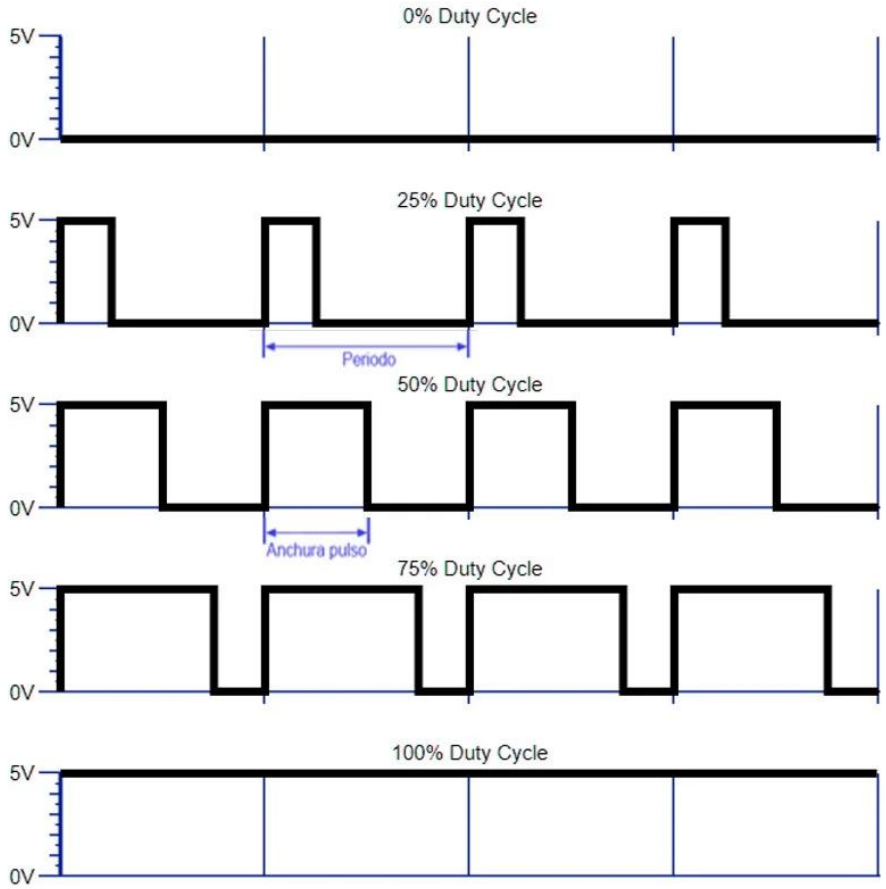
- Leds
- Motores DC
- Motores paso a paso
- Servos
- Buzzers
- Transistores





Arquitectura de Microcontroladores

Puertos de Salida PWM



Arquitectura de Microcontroladores

Puertos de Salida PWM

Fabricar una **salida digital** es muy **fácil y barato**, fabricar una salida analógica no lo es tanto.

Podemos emular una salida analógica, alternando el estado del pin digital muy rápido y el valor que entrega es el **promedio**.

PWM se utiliza para controlar la **intensidad** de una señal analógica mediante la **variación de la duración y el ancho** de los pulsos de una señal digital. (**no es analógica continua**).

Se puede conectar cualquier cosa a la cual se le pueda o deba variar la potencia.

Podemos conectar un led que en vez de parpadear, aumente y disminuya su intensidad, un servo, un buzzer (chicharra).

También un Motor DC usando el shield correspondiente.

Arquitectura de Microcontroladores

Puertos de Salida PWM

Configuración:
```
pinMode(PIN_LED, OUTPUT);
pinMode(PIN_POTE, INPUT)
```

Uso:
```
analogWrite(PIN_LED, 128);

analogWrite(PIN_LED, analogRead(PIN_POTE));

analogWrite(PIN_LED, map(analogRead(PIN_POTE), 0, 2047, 0, 255));
```

El duty cycle (0-100%) se representa como un número entre 0 y 255.

Arquitectura de Microcontroladores

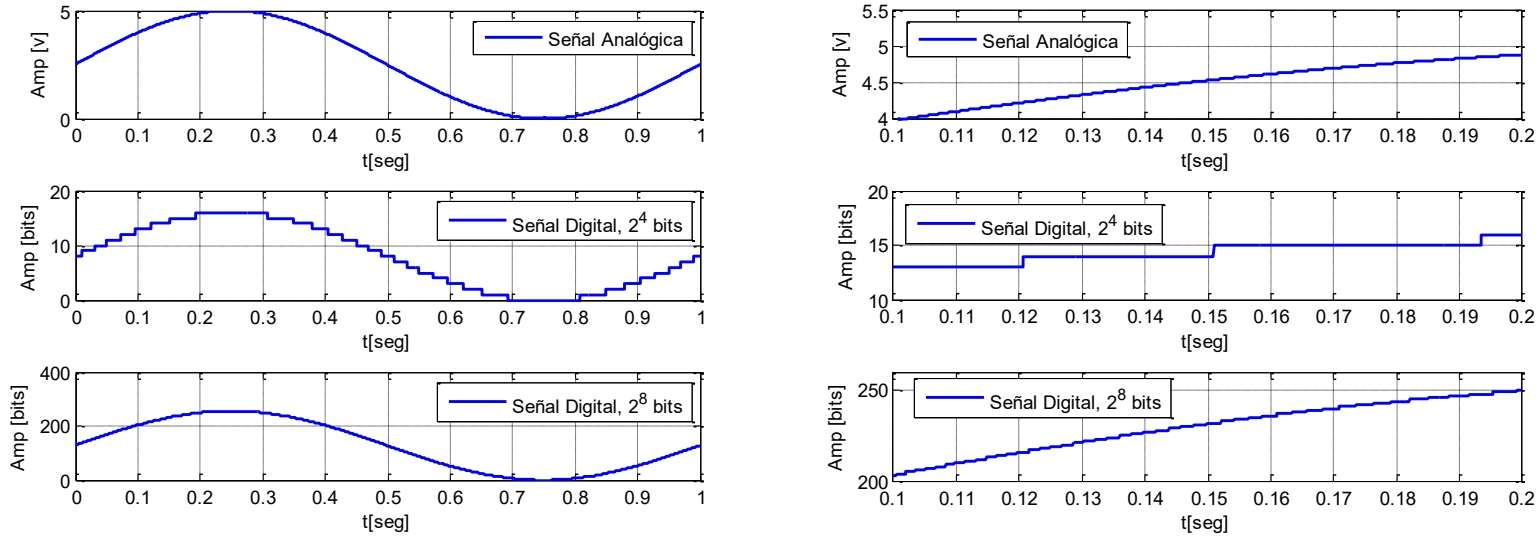
Puertos de entrada analógica (ADC)

Una entrada **analógica** es una que le permite al microcontrolador leer un **voltaje variable**, típicamente desde los 0V hasta el voltaje de alimentación del microcontrolador.

Un Puerto de entrada analógica convierte un valor de tensión en un número entero.

Arquitectura de Microcontroladores

Puertos de entrada analógica



La cantidad de bits de la conversión tiene directa relación con la calidad de la señal digitalizada. Con 4 bits la señal puede tomar 16 valores diferentes. Con 8 bits, 256 valores diferentes. Soporta hasta 12 bits, es decir 4096 valores.

Arquitectura de Microcontroladores

Puertos de entrada analógica

Uso: `analogRead(34);`

Podemos configurar la cantidad de bits utilizados mediante…

```
analogReadResolution(10)
```

```
float voltaje = (analogRead(34)/1024)*3.3
```

Arquitectura de Microcontroladores

Puertos de entrada analógica

AnalogRead(nombreDelPin), devuelve un número (que depende de la cantidad de bits asignados)

A eso le tenemos que dar alguna característica (qué representa?).

(Nosotros vamos a poner un potenciómetro para poder medir valores analógicos)

Valores por defecto de resolución: para analogRead() es 12 (0 a 4095)

para analogWrite() es 8 (0 a 255)

Arquitectura de Microcontroladores

Puertos de Comunicaciones

Para que dos dispositivos se comuniquen, sean computadoras, microcontroladores, etc, se necesita un método (protocolo) de comunicación acordado de antemano. La comunicación serie es una de las formas más comunes de comunicación entre dispositivos digitales.

Para que puedan comunicarse, dos dispositivos tienen que acordar ciertas cosas.

- La velocidad a la cual los datos serán enviados y leídos.
- Los niveles de voltaje representando a un 1 y a un 0.

La mayoría de los procesadores vienen equipados con una o más UART. (**Universal Asynchronous Receiver-Transmitter)**

Arquitectura de Microcontroladores

Puertos de Comunicaciones

En ESP32, estos conceptos son transparentes para el usuario.

Lo único que hay que tener en cuenta es:

- La velocidad de transmisión (baud rate) debe ser igual en el receptor y en el transmisor.
- La comunicación serie transmite bytes.
- Los números de más de un byte (ej, punto flotante) son transformados a caracteres y luego transmitidos.

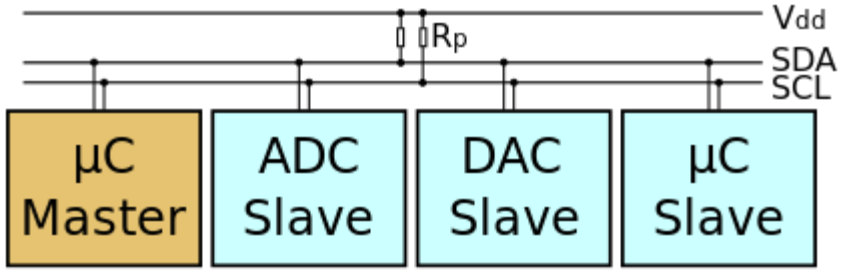
Ejemplos de UARTs:

- Puerto Serie
- Puerto USB

Arquitectura de Microcontroladores

I²C:

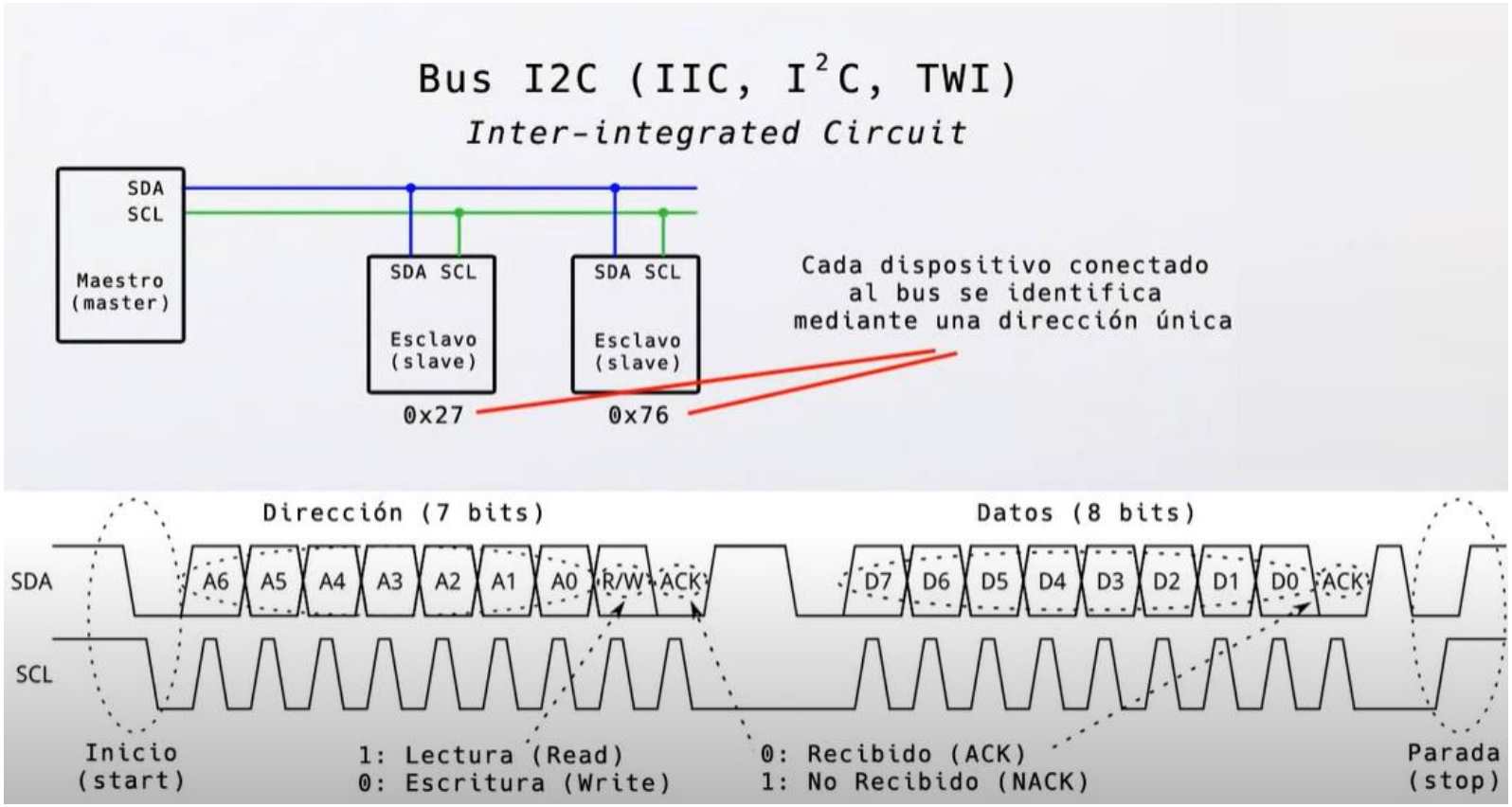
- En el diagrama se encuentran representados tres dispositivos. El I²C precisa de dos líneas de señal: reloj (SCL, Serial Clock) y la línea de datos (SDA, Serial Data).
- La dirección de I²C estándar es el primer byte enviado por el maestro, aunque los primeros 7 bits representan la dirección y el octavo bit (R/W-Bit) es el que comunica al esclavo si debe recibir datos del maestro o enviar datos al maestro.
- Cada uno de los circuitos integrados con capacidad de soportar un I²C tiene una dirección predeterminada por el fabricante (¡Atención con las superposiciones!).



Arquitectura de Microcontroladores

Fuente:
https://www.youtube.com/watch?v=kuLgPLrg-cY

I²C :



Arquitectura de Microcontroladores

SPI:

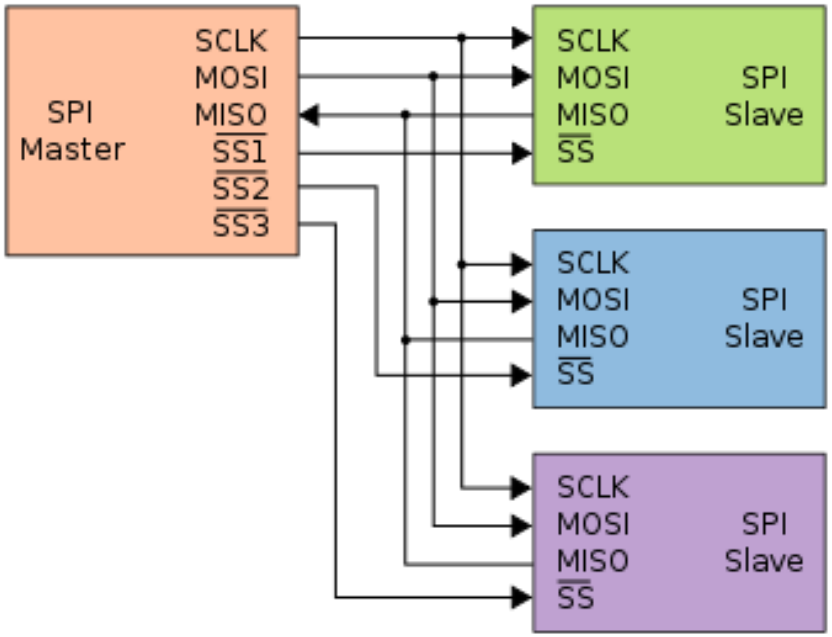
El **Bus SPI** (del inglés *Serial Peripheral Interface*) es un estándar de comunicaciones, usado principalmente para la transferencia de información entre circuitos integrados en equipos electrónicos. El bus de interfaz de periféricos serie o bus SPI es un estándar para controlar casi cualquier dispositivo electrónico digital que acepte un flujo de bits serie regulado por un reloj (comunicación sincrónica).

La sincronización y la transmisión de datos se realiza por medio de 4 señales:

- **CLK** *(Clock)*: Es el pulso que marca la sincronización. Con cada pulso de este reloj, se lee o se envía un bit. También llamado TAKT (en alemán).
- **MOSI** *(Master Output Slave Input)*: Salida de datos del Master y entrada de datos al Slave. También llamada SIMO.
- **MISO** *(Master Input Slave Output)*: Salida de datos del Slave y entrada al Master. También conocida por SOMI.
- **SS/Select**: Para seleccionar un Slave, o para que el Master le diga al Slave que se active. También llamada SSTE. Una conexión SS por dispositivo

Arquitectura de Microcontroladores

SPI:



Arquitectura de Microcontroladores

Con **UART** puedo conectar típicamente **sólo dos dispositivos** entre sí.

Con **I2C** hacen falta **sólo dos conexiones** para datos para **cualquier cantidad** de dispositivos. (teóricamente 127 esclavos).

Con **SPI** necesito **4 conexiones de datos** para conectar **un solo esclavo**, debo disponer de **otra conexión** para cada esclavo **adicional**.

Consejos

- Sobre la Electrónica y la temperatura de los componentes

  Tocar los componentes. Si están muy calientes algo anda mal.

- Sobre el uso de los Servos

  Los servos deben ser alimentados desde fuente externa.

Más Información

https://www.arduino.cc

https://www.arduino.cc/en/Reference/HomePage

https://www.arduino.cc/en/ArduinoCertified/IntelGalileo

Referencias

https://learn.sparkfun.com/tutorials/galileo-getting-started-guide

https://ccrma.stanford.edu/wiki/Microcontroller_Architecture#Registers_on_the_AVR_Microcontroller

http://www.pyroelectro.com/tutorials/fading_led_pwm/theory.html

https://en.wikipedia.org/wiki/Pulse-width_modulation

https://itp.nyu.edu/physcomp/lessons/microcontrollers/

https://www.luisllamas.es/leer-un-pulsador-con-arduino/

https://es.wikipedia.org/wiki/I%C2%B2C

https://es.wikipedia.org/wiki/Serial_Peripheral_Interface

https://www.arduino.cc/en/ArduinoCertified/IntelGalileo